

ФАЙЛОВЕ И ОБРАБОТКА С ИЗКЛЮЧЕНИЯ

Файлът е колекция от данни, съхранявани в едно устройство, идентифицирани с име. Това може да бъде текстов документ, картина/изображение, аудио, аудио-видео или друга колекция от данни. Общият формат/разширения за текстови документи са .doc, .docx (документи на Microsoft word), odt (текст на отворен документ в Libre Office), .pdf (формат за преносим документ на Adobe), rtf (формат на богат текст на Microsoft), .tex (LaTeX текст), .txt (текст на Microsoft Notepad). Файловете формати с изображения са: .jpg, .tiff, .gif, .png, bmp. Често използваните аудио формати са: .wav и .mp3. Аудио-видео форматът включва .avi, .mp4, .mkv, .mov, .flv, .wmv и др..

Python поддържа обработка на файлове и позволява на потребителите да обработват файлове, т.е. да четат и записват файлове, заедно с много други опции за обработка на файлове.

Файловите функции на Python `open()` и `close()` се използват за отваряне и затваряне на файл.

Функциите `read()` и `write()` се използват за четене и записване на текст (или числови или двоични данни) съответно от/във файла.

Режимите на отваряне на файлове в Python са: `r` (четене), `w` (запис), `a` (добавяне), `rb` (четене на двоични данни), `wb` (запис на двоични данни).

ИЗХОД С ИЗПОЛЗВАНЕ НА ФУНКЦИЯТА PRINT()

За да извеждане на данните на екрана, се използва функцията за `print()`. След написване на `print(argument)` и ще се отпечата аргумента в следващия ред, след като се натисне клавиша ENTER.

Дефиниции:

- (1) Аргументът е стойност, който се предава на функция, при нейното извикване.
 - (2) Стойността е буква или цифра.
 - (3) Променливата е име, което се отнася до стойността и започва с литерал.
 - (4) Инструкцията за присвояване създава нови променливи и им дава стойности.
- Този синтаксис е валиден както в Python 3.x, така и в Python 2.x.

Пример: Да се разпечатат данните - “Guido” на екрана.

За целта се поставят в скоби - `print("Guido")`.

```
>>>print("Guido")  
Guido
```

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

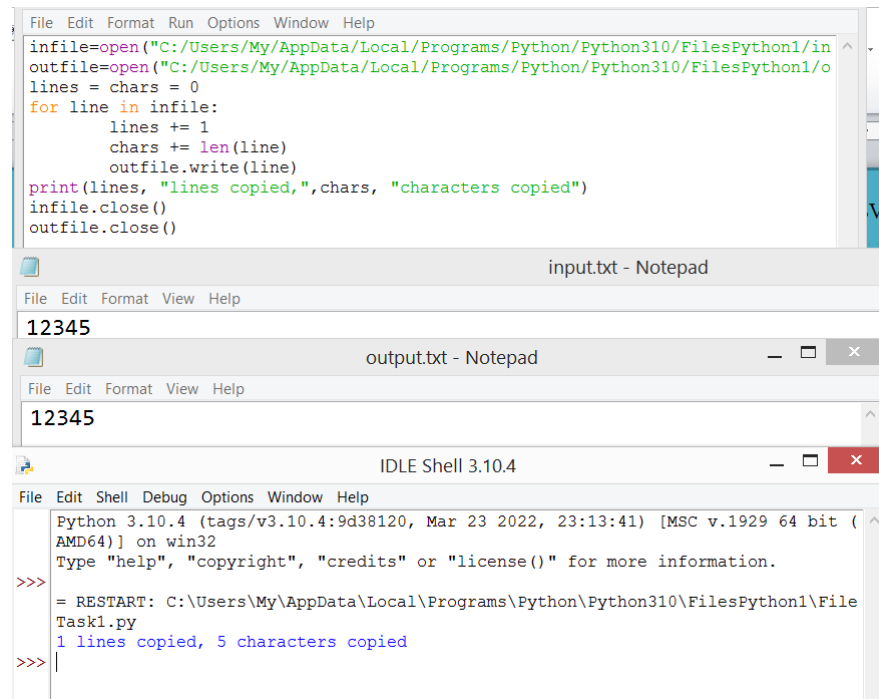
- reading и writing text file

Пример: Да се копира съдържанието на текстов файл (input.txt) да се копира в друг текстов файл (output.txt)

#FileTaks1

File copy – content of a text file (input.txt) is copied to another text file (output.txt)

```
infile=open("C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/input.txt","r")
outfile=open("C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/output.txt","w")
lines = chars = 0
for line in infile:
    lines += 1
    chars += len(line)
    outfile.write(line)
print(lines, "lines copied",chars, "characters copied")
infile.close()
outfile.close()
```



The screenshot displays a Python IDE with three windows. The top window shows the Python script for copying a file. The middle window shows the Notepad application with 'input.txt' containing the text '12345' and 'output.txt' containing the same text. The bottom window shows the IDLE Shell with the execution output.

```
File Edit Format Run Options Window Help
infile=open("C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/in
outfile=open("C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/o
lines = chars = 0
for line in infile:
    lines += 1
    chars += len(line)
    outfile.write(line)
print(lines, "lines copied",chars, "characters copied")
infile.close()
outfile.close()
```

input.txt - Notepad

12345

output.txt - Notepad

12345

IDLE Shell 3.10.4

```
File Edit Shell Debug Options Window Help
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\My\AppData\Local\Programs\Python\Python310\FilesPython1\File
Task1.py
1 lines copied, 5 characters copied
>>> |
```

Резултат →

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

- reading и writing numeric data file

Пример: Да се копират всички всички числа от входния файл (input.dat) и да се запишат в изходен файл (output.dat).

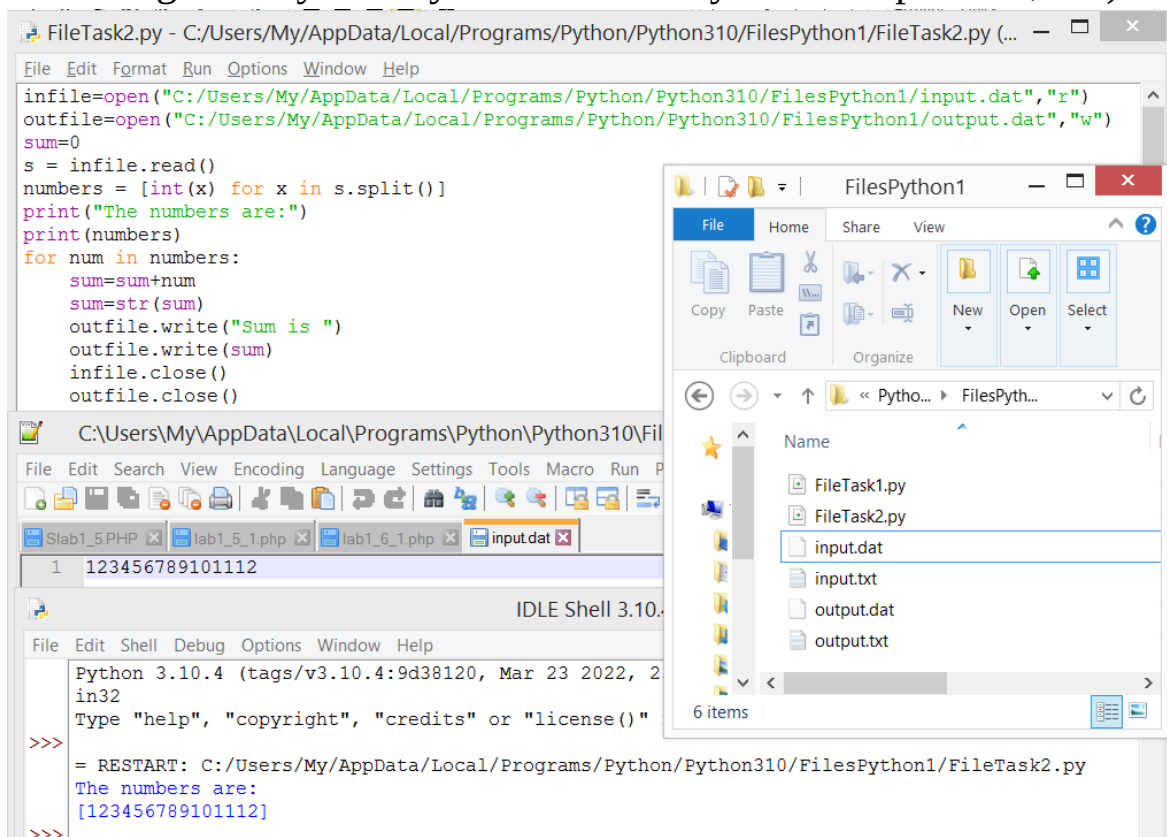
#FileTaks2

```
infile=open("C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/input.dat","r")
outfile=open("C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/output.dat","w")
sum=0
s = infile.read()
numbers = [int(x) for x in s.split()]
print("The numbers are:")
print(numbers)
    for num in numbers:
        sum=sum+num
    sum=str(sum)
outfile.write("Sum is ")
outfile.write(sum)
infile.close()
outfile.close()
```

Резултат:

The numbers are:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]



ОБРАБОТКА НА ИЗКЛЮЧЕНИЯТА (EXCEPTION HANDLING)

Изключения са аномалии по време на изпълнение или необичайни условия, които програмата среща по време на своето изпълнение, като деление на нула (ZeroDivisionError), отваряне на файл за четене, който не съществува (IOError), отстъпът не е посочен правилно (IndentationError) и т.н. Но като цяло, изключението нарушава нормалния поток на изпълнение. Обработката на изключения позволява на програмата да се справя с изключенията и да продължи нормалното си изпълнение. Изявлението **try** в Python може да има повече от една клауза за изключение за обработка на различни изключения. То може също да има незадължителен оператор **else** и/или **finally**.

Синтаксисът **try-except** е:

try:

<body>

except <ExceptionType1>:

<handler1>

...

except <ExceptionTypeN>:

<handlerN>

except: else:

<handlerExcept>

<process_else>

finally:

<process_finally>

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

Множеството изключения са подобни на `elifs` в Python. Когато възникне изключение, се проверява дали съответства на изключение в клауза за изключение след клаузата `try` едно по едно (последователно). Ако бъде намерено съвпадение, манипулаторът за съответния случай се изпълнява и останалите клаузи за изключение се пропускат. Имайте предвид, че `<ExceptionType>` в последната клауза с изключение може да бъде пропусната. Ако изключението не съвпада с нито един от типовете изключения преди последната клауза за изключение, се изпълнява `<handlerExcept>` за последната клауза за изключение. Изявлението `try` може да има незадължителна клауза `else`, която се изпълнява, ако не е повдигнато изключение в тялото на `try`. Изявлението `try` може да има незадължителна клауза `finally`, която има за цел да дефинира действия за почистване, които трябва да се извършват при всякакви обстоятелства.

#FileTask3:

#Handling exceptions that occurs at runtime such as division by zero, syntax error and #raising and handling the exception.
`try:`

```
    number1, number2 = eval(input("Enter two numbers separated by a comma: "))
    result = number1 / number2
    print("Result is", result)
    if(number1==0):
        raise RuntimeError()
except ZeroDivisionError:
    print("Division by Zero")
except SyntaxError:
    print("A comma may be Missing in the Input")
except RuntimeError:
    print("May be Meaningless")
except:
    print("Something Wrong in the Input")
else:
    print("No Exceptions")
finally:
    print("Finally Clause is Executed")
```

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

При зададените входни данни от 1) до 5) ще бъдат изведени следните резултати:

1) Enter two numbers separated by a comma: 4, 0

Division by Zero

Finally Clause is Executed

>>>

2) Enter two numbers separated by a comma: 5 6

A comma may be Missing in the Input

Finally Clause is Executed

>>>

3) Enter two numbers separated by a comma: 0, 9

Result is 0.0

May be Meaningless

Finally Clause is Executed

>>>

4) Enter two numbers separated by a comma: 66

Something Wrong in the Input

Finally Clause is Executed

>>>

5) Enter two numbers separated by a comma: 12,4

Result is 3.0 No Exceptions

Finally Clause is Executed

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

ВЪВЕЖДАНЕ И ИЗХОД НА ТЕКСТ

Отварянето на файл създава файлов обект.

В този пример променливата `f` се отнася до новия файлов обект.

```
f = open("test.dat","w")  
print (f)  
<_io.TextIOWrapper name='test.dat' mode='w' encoding='cp1251'>
```

Функцията `open` приема два аргумента. Първият е името на файла, а вторият е режимът - "w", който означава, че файлът се отваря за запис. **Ако няма файл с име `test.dat`, той ще бъде създаден. Ако вече има такъв, той ще бъде заменен от файла, който пишем.** Когато отпечатаме файловия обект, виждаме името на файла, режима и местоположението на обекта.

#FileTaks4

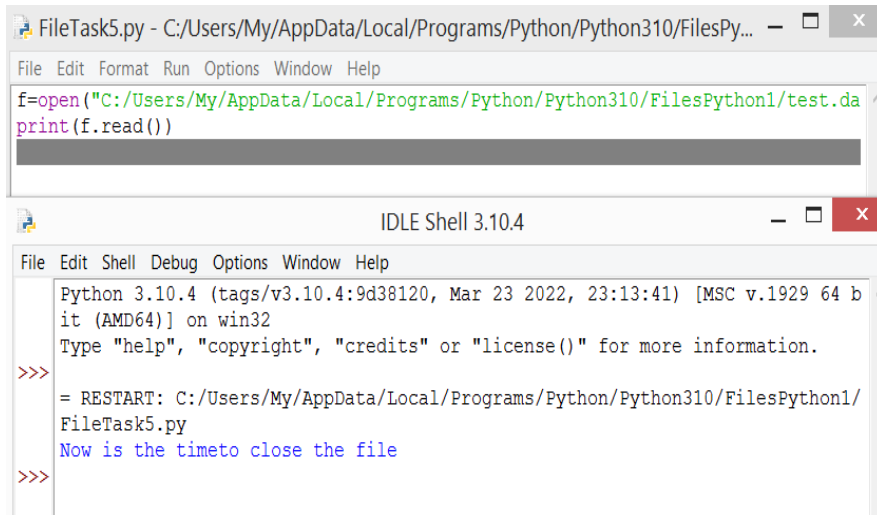
```
f = open("test.dat","w")  
print (f)  
f.write("Now is the time")  
f.write("to close the file")  
f.close()
```

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

Отварянето на файла за четене става чрез промяна на аргументът на режима с "r":

#FileTaks5

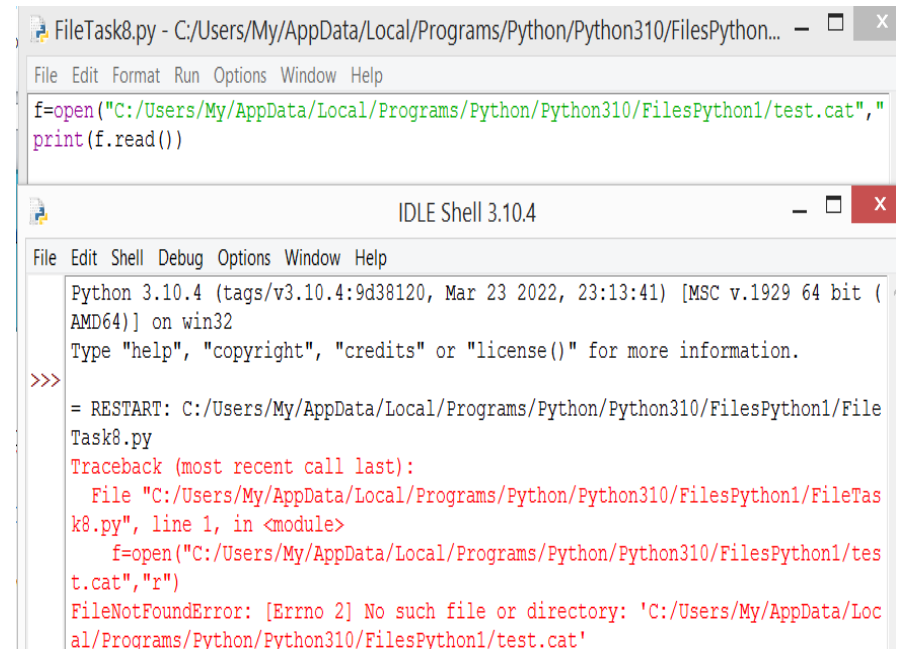
```
f=open("C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/test.dat","r")
print(f.read())
```



The screenshot shows the Python IDLE environment. The top window, titled 'FileTask5.py', contains the code: `f=open("C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/test.da", "r")` and `print(f.read())`. The bottom window, titled 'IDLE Shell 3.10.4', shows the output of the program: `Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32`, followed by a prompt `>>>` and the text `= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/FileTask5.py` and `Now is the timeto close the file`.

При опит да се отвори несъществуващ файл се получава грешка:

```
>>> f = open("test.cat","r")
```



The screenshot shows the Python IDLE environment. The top window, titled 'FileTask8.py', contains the code: `f=open("C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/test.cat", "r")` and `print(f.read())`. The bottom window, titled 'IDLE Shell 3.10.4', shows the output of the program: `Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32`, followed by a prompt `>>>` and the text `= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/FileTask8.py`. Below this, a `Traceback (most recent call last):` is shown, indicating the error: `File "C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/FileTask8.py", line 1, in <module>` `f=open("C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/test.cat", "r")` `FileNotFoundError: [Errno 2] No such file or directory: 'C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/test.cat'`.

ОБРАБОТКА НА ИЗКЛЮЧЕНИЯ – ПРОГРАМНИ ГРЕШКИ И ОБРАБОТКА НА ИЗКЛЮЧЕНИЯТА

Видове програмни грешки:

- Синтактични, семантични и логически грешки
- Грешки по време на компилиране и изпълнение
- Тест на драйвъри
- Техники за отстраняване на грешки
- Обработка на изключения
- Най-често срещани видове изключения
- Клаузата throws и изявлението throw
- Улавяне на изключения чрез конструкцията try-catch
- Разпространение на изключения
- Изключения при четене от файл

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

Според типа на грешката:

- **Синтактични грешки:** грешки поради факта, че синтаксисът на езика не се спазва.
- **Семантични грешки:** грешки, дължащи се на неправилно използване на програмни изрази.
- **Логически грешки:** грешки поради факта, че спецификацията не се спазва.

Според това, КОГА са открити грешките биват:

- **Грешки по време на компилиране:** синтактични грешки и статични семантични грешки, посочени от компилатора.
- **Грешки по време на изпълнение:** динамични семантични грешки и логически грешки, които не могат да бъдат открити от компилатора.
- **Синтактични грешки:** Синтаксичните грешки се дължат на факта, че синтаксисът на езика Java не се спазва.
- **Семантични грешки:** Семантичните грешки показват неправилно използване на Java изрази.
- **Логически грешки:** Логическите грешки са причинени от факта, че спецификацията на софтуера не се спазва. Програмата се компилира и изпълнява без грешки, но не генерира искания резултат.

ВИДОВЕ ИЗКЛЮЧЕНИЯ (TYPES OF EXCEPTION CLASS)

- **Base Exception**
 - **Exception**
 - ❖ **Arithmetic Error**
- **Floating Point Error**
- **Overflow Error**
- **Zero Division Error**
- **Assertion Error**

▪ **Base Exception**

Класът Base Exception, както подсказва името е основният клас за всички вградени изключения в Python. Обикновено това изключение никога не се повдига самостоятелно, а вместо това трябва да бъде наследено от други, по-малки класове изключения, които могат да бъдат повдигнати.

□ **Exception**

Изключението е най-често наследяваният тип изключение (извън истинския базов клас на Base Exception). В допълнение, всички класове изключения, които се считат за грешки, са подкласове на класа Exception. По принцип всеки персонализиран клас изключения, който се създава в кода ни, трябва да се наследява от Exception.

❖ **Arithmetic Error**

Базовият клас за разнообразието от аритметични грешки като: при опит за делене на нула или аритметичен резултат, който е твърде голям, за да може Python да го представи точно.

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

▪ Assertion Error

Тази грешка се появява, когато извикването на оператора [assert] не успее.

BINARY I/O USING PICKLE — PYTHON OBJECT SERIALIZATION

Модулът pickle реализира двоични протоколи за сериализиране и десериализиране на обектна структура на Python.

„Pickling“ е процесът, при който йерархията на обектите на Python се преобразува в поток от байтове, а „unpickling“ е обратната операция, при която поток от байтове (от двоичен файл или подобен на байтов обект) се преобразува обратно в йерархия на обекти.

Pickling (и unpickling) е алтернативно известно още като „serialization“, „marshalling“ 1 или „flattening“; но, за да се избегне объркване, термините, използвани тук, са pickling” и “unpickling”.

▪ Module Interface

За да **сериализирате** йерархия на обекти, просто извиквате функцията **dumps()**. По същия начин, за да **десериализирате** поток от данни, извиквате функцията **loads()**. Въпреки това, ако искате **повече контрол върху сериализацията и десериализацията**, можете да създадете съответно обект **Pickler** или **Unpickler**.

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

Модулът `pickle` осигурява следните константи:

`pickle.HIGHEST_PROTOCOL`

Цяло число, най-високата налична версия на протокола. Тази стойност може да бъде предадена като стойност на протокола на функциите `dump()` и `dumps()`, както и на конструктора `Pickler`.

`pickle.DEFAULT_PROTOCOL`

Цяло число, версията на протокола по подразбиране, използвана за `pickling`. Може да е по-малко от `HIGHEST_PROTOCOL`. В момента протоколът по подразбиране е 4, въведен за първи път в Python 3.4 и несъвместим с предишни версии.

Модулът за `pickle` предоставя следните функции, за да направи процеса `pickling` по-удобен:

`pickle.dump(obj, file, protocol=None, *, fix_imports=True, buffer callback=None)`

Напишете pickled представяне на обекта `obj` в отворения файл обектен файл. Това е еквивалентно на `Pickler(file, protocol).dump(obj)`.

READING AND WRITING TO A BINARY FILE

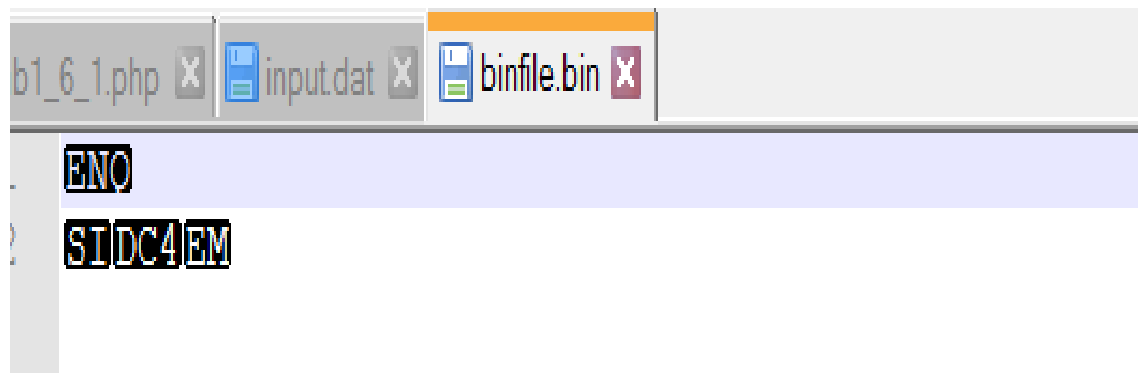
Функцията `open()` отваря файл в текстов формат по подразбиране. *За да отворите файл в двоичен формат, добавете 'b' към параметъра за режим.* Следователно **режимът "rb" отваря файла в двоичен формат за четене, докато режимът "wb" отваря файла в двоичен формат за запис.** За разлика от файловете в текстов режим, двоичните файлове не са четими от човека. Когато се отвори с който и да е текстов редактор, данните са неразпознаваеми.

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

Следният код съхранява списък с числа в двоичен файл. Списъкът първо се преобразува в байтов масив, преди да се запише. Вградената функция `byte array()` връща байтово представяне на обекта.

#FileTaks6: Write to a Binary File

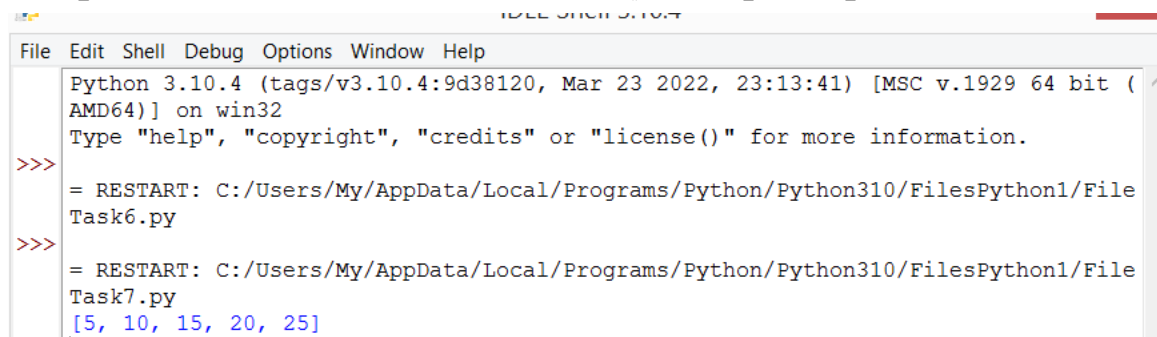
```
f=open("binfile.bin", "wb")
num=[5, 10, 15, 20, 25]
arr=bytearray(num)
f.write(arr)
f.close()
```



За да прочетете горния двоичен файл, изходът на метода `read()` се прехвърля към списък с помощта на функцията `list()`.

#FileTaks7: : Reading a Binary File

```
f=open("binfile.bin", "rb")
num=list(f.read())
print (num)
f.close ()
```



СЛУЧАИ: ПРЕБРОЯВАНЕ НА ЗНАК ВЪВ ФАЙЛ

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

#FileTaks7: Напишете програма на Python, която да изброи знаците от текстов файл.

Вход: abc.txt

German Unity Day: The Day of German Unity (German: Tag der Deutschen Einheit) is the national day of Germany, celebrated on 3 October as a public holiday. It commemorates the anniversary of German reunification in 1990, when the goal of a united Germany that originated in the middle of the 19th century, was fulfilled again. Therefore, the name addresses neither the re-union nor the union, but the unity of Germany. The Day of German Unity on 3 October has been the German national holiday since 1990, when the reunification was formally completed.

Елементарно решение:

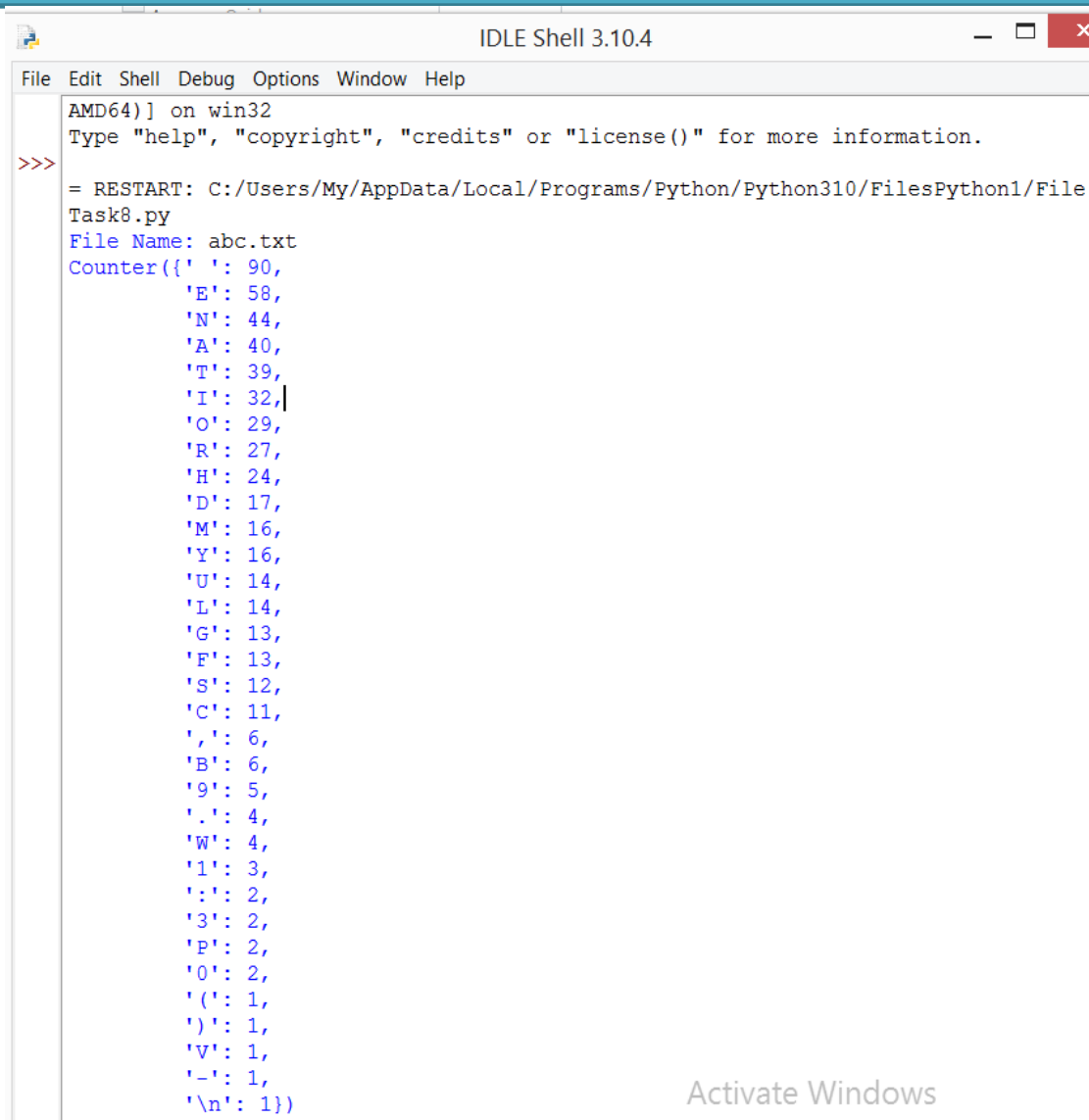
Код:

```
import collections
import pprint
file_input = input('File Name: ')
with open(file_input, 'r') as info:
    count = collections.Counter(info.read().upper())
    value = pprint.pformat(count)
print(value)
```

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

Резултат:

File Name: **abc.txt**



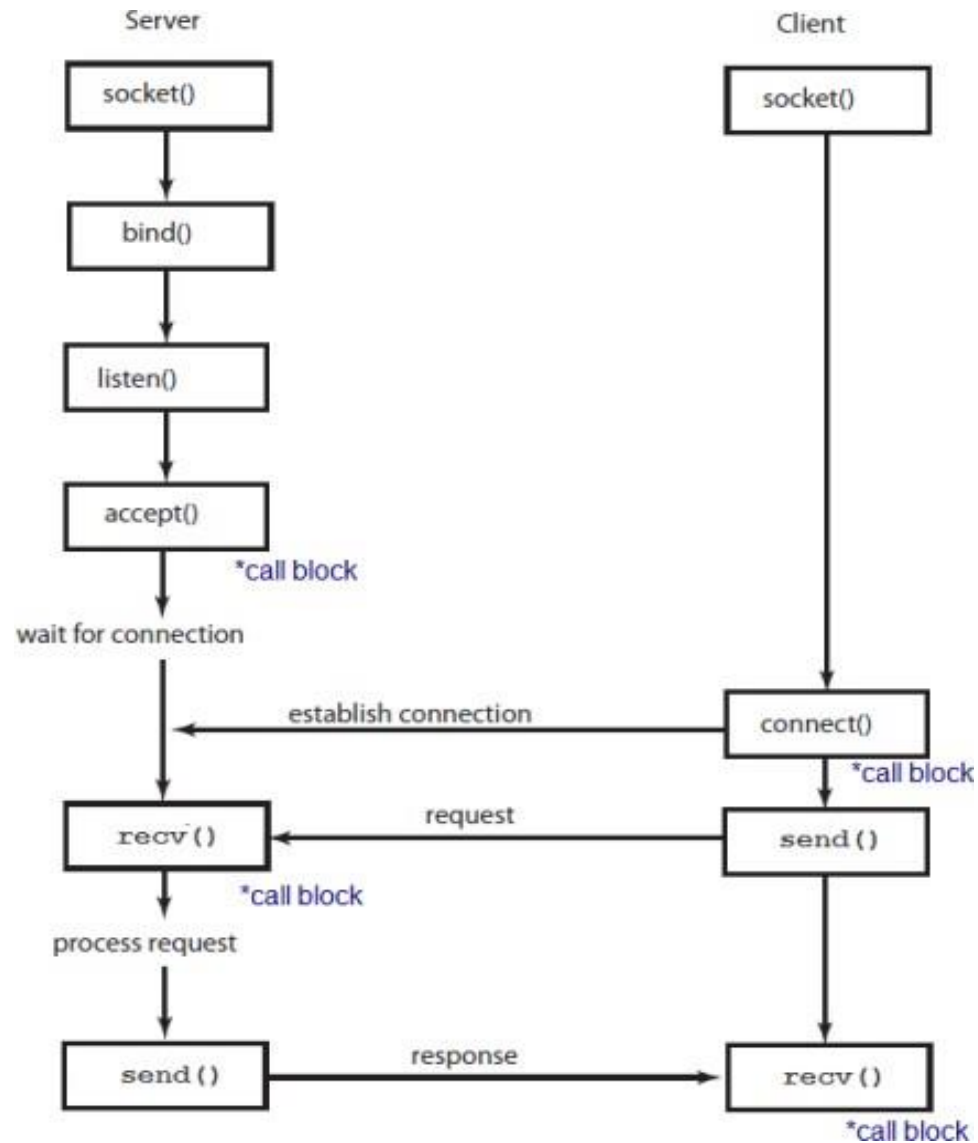
```
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/File
Task8.py
File Name: abc.txt
Counter({' ': 90,
        'E': 58,
        'N': 44,
        'A': 40,
        'T': 39,
        'I': 32,
        'O': 29,
        'R': 27,
        'H': 24,
        'D': 17,
        'M': 16,
        'Y': 16,
        'U': 14,
        'L': 14,
        'G': 13,
        'F': 13,
        'S': 12,
        'C': 11,
        ',': 6,
        'B': 6,
        '9': 5,
        '.': 4,
        'W': 4,
        'l': 3,
        ':': 2,
        '3': 2,
        'P': 2,
        '0': 2,
        '(': 1,
        ')': 1,
        'V': 1,
        '-': 1,
        '\n': 1})
```

Activate Windows

КЛИЕНТ СЪРВЪР АРХИТЕКТУРА В PYTHON

- 1. `socket.socket()`:** Създайте нов сокет, използвайки даденото семейство адреси, тип сокет и номер на протокола.
- 2. `socket.bind(address)`:** Свържете сокета с адрес.
- 3. `socket.listen(backlog)`:** Слушайте за връзки, направени към socket-a. Аргументът **backlog** определя максималния брой връзки на опашката и трябва да бъде най-малко 0; максималната стойност е системно-зависима (обикновено 5), минималната стойност е сведена до 0.
- 4. `socket.accept()`:** Връщаната стойност е двойка (**conn**, **address**), където **conn** е нов обект на сокет, който може да се използва за изпращане и получаване на данни за връзката, а **address** е адресът, свързан към сокета от другия край на връзката. При **accept()** се създава нов сокет, който е различен от посочения сокет. Този нов сокет се използва единствено за комуникация с този конкретен клиент. За TCP сървъри, обектът сокет, използван за получаване на връзки, не е същият сокет, използван за извършване на последваща комуникация с клиента. По-специално, системното извикване **accept()** връща нов обект на сокет, който всъщност се използва за връзката. Това позволява на сървъра да управлява връзки от голям брой клиенти едновременно.
- 5. `socket.send(bytes[, flags])`:** Изпращане на данни към сокета. Контактът трябва да бъде свързан към отдалечен контакт. Връща броя изпратени байтове. Приложенията са отговорни за проверката дали всички данни са изпратени; ако са предадени само някои от данните, приложението трябва да опита да достави останалите данни.
- 6. `socket.close()`:** Маркирайте сокета затворен, като всички бъдещи операции върху обекта **socket** ще се провалят. Отдалеченият край няма да получава повече данни (след като данните от опашката бъдат изтрети). Сокетите се затварят автоматично, когато се събира боклука, но е препоръчително да ги затворите с **close()** изрично.

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул



ПРОГРАМИРАНЕ НА КЛИЕНТ СЪРВЪР СОКЕТ

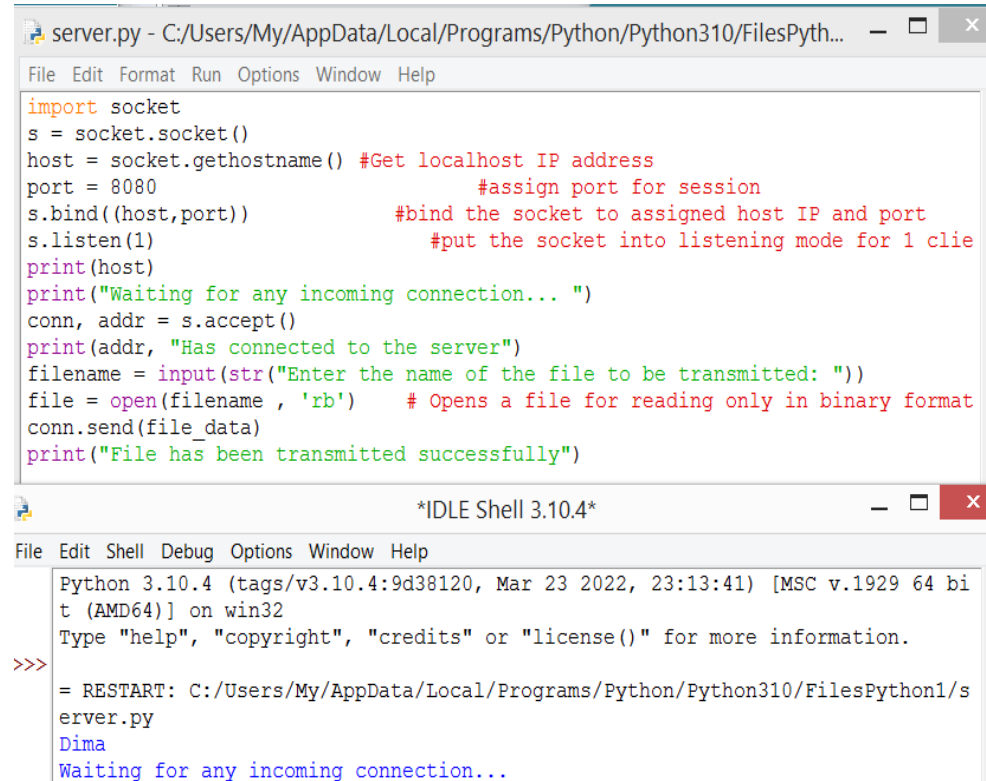
Procedure (Server): Сървърът ще хоства мрежата, използвайки назначения хост и порт, чрез който клиентът ще се свърже със сървъра. Сървърът ще действа като подател и потребителят ще трябва да въведе името на файла, който той/тя иска да предаде. Потребителят трябва да се увери, че файлът, който трябва да бъде изпратен, е в същата директория като програмата "server.py".

Procedure (Client): Клиентската програма ще подкани потребителя да въведе хост адреса на сървъра, докато портът вече ще бъде присвоен на променливата на порта. След като клиентската програма се свърже със сървъра, тя ще поиска от потребителя име на файл, което да се използва за файла, който ще бъде получен от сървъра. Накрая клиентската програма ще получи файла и ще го остави в същата директория под същото име на файл, зададено като потребителя.

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

Server Coding:

```
import socket
s = socket.socket()
#Get localhost IP address
host = socket.gethostname()
#assign port for session
port = 8080
#bind the socket to assigned host IP and port
s.bind((host,port))
#put the socket into listening mode for 1 client
s.listen(1)
print(host)
print("Waiting for any incoming connection... ")
conn, addr = s.accept()
print(addr, "Has connected to the server")
filename = input(str("Enter the name of the file to be transmitted: "))
# Opens a file for reading only in binary format file_data = file.read(1024)
file = open(filename , 'rb')
conn.send(file_data)
print("File has been transmitted successfully")
```



The screenshot shows a Python IDE window titled 'server.py - C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython...'. The code in the editor is the same as the one on the left. Below the editor is the 'IDLE Shell 3.10.4*' window, which shows the output of the program. The shell displays the Python version and architecture, followed by the prompt 'Dima' and the message 'Waiting for any incoming connection...'. The shell also shows the command prompt 'C:\Users\My\AppData\Local\Programs\Python\Python310\FilesPython1\s' and the file 'server.py'.

```
server.py - C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython...
File Edit Format Run Options Window Help

import socket
s = socket.socket()
host = socket.gethostname() #Get localhost IP address
port = 8080                 #assign port for session
s.bind((host,port))         #bind the socket to assigned host IP and port
s.listen(1)                 #put the socket into listening mode for 1 client
print(host)
print("Waiting for any incoming connection... ")
conn, addr = s.accept()
print(addr, "Has connected to the server")
filename = input(str("Enter the name of the file to be transmitted: "))
file = open(filename , 'rb') # Opens a file for reading only in binary format
conn.send(file_data)
print("File has been transmitted successfully")

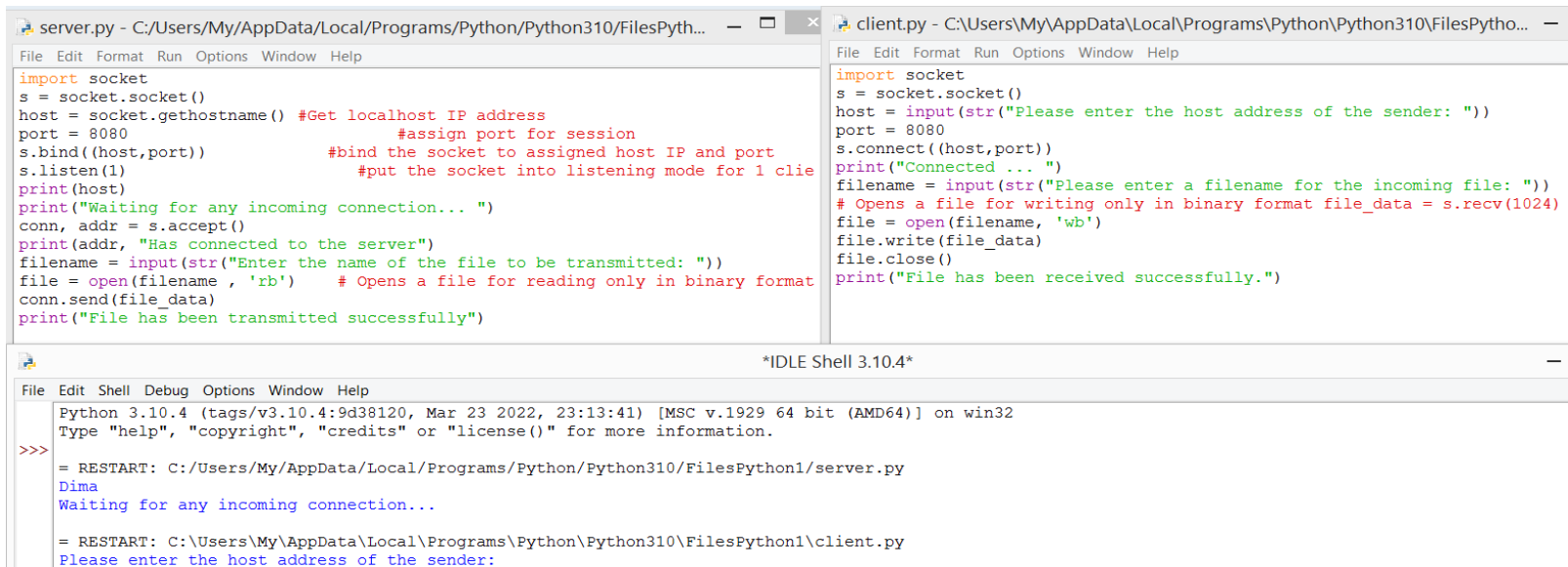
*IDLE Shell 3.10.4*
File Edit Shell Debug Options Window Help

Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bi
t (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/s
erver.py
Dima
Waiting for any incoming connection...
```

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

Client Coding:

```
import socket
s = socket.socket()
host = input(str("Please enter the host address of the sender: "))
port = 8080
s.connect((host,port)) print("Connected ... ")
filename = input(str("Please enter a filename for the incoming file: "))
# Opens a file for writing only in binary format file_data = s.recv(1024)
file = open(filename, 'wb')
file.write(file_data)
file.close()
print("File has been received successfully.")
```



The screenshot displays a Python IDE with three windows. The top-left window, titled 'server.py', contains the following code:

```
import socket
s = socket.socket()
host = socket.gethostname() #Get localhost IP address
port = 8080 #assign port for session
s.bind((host,port)) #bind the socket to assigned host IP and port
s.listen(1) #put the socket into listening mode for 1 client
print(host)
print("Waiting for any incoming connection... ")
conn, addr = s.accept()
print(addr, "Has connected to the server")
filename = input(str("Enter the name of the file to be transmitted: "))
file = open(filename, 'rb') # Opens a file for reading only in binary format
conn.send(file_data)
print("File has been transmitted successfully")
```

The top-right window, titled 'client.py', contains the following code:

```
import socket
s = socket.socket()
host = input(str("Please enter the host address of the sender: "))
port = 8080
s.connect((host,port))
print("Connected ... ")
filename = input(str("Please enter a filename for the incoming file: "))
# Opens a file for writing only in binary format file_data = s.recv(1024)
file = open(filename, 'wb')
file.write(file_data)
file.close()
print("File has been received successfully.")
```

The bottom window, titled '*IDLE Shell 3.10.4*', shows the command prompt output:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/server.py
Dima
Waiting for any incoming connection...
= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/client.py
Please enter the host address of the sender:
```

TCP И UDP В ТРАНСПОРТНИЯ СЛОЙ

Слой 3 или мрежовият слой използва IP или интернет протокол, който като протокол без връзка, третира всеки пакет поотделно, което води до липса на надеждност по време на предаване.

Например: когато данните се изпращат от един хост към друг, всеки пакет може да поеме различен път, дори ако принадлежи към една и съща сесия. Това означава, че пакетите може или не може да пристигнат в правилния ред. Следователно IP разчита на протоколите от по-висок слой, за да осигури надеждност.

- **TCP (Transmission Control Protocol):**

Протокол за управление на предаването - Transmission Control Protocol (TCP) – комуникационен протокол, ориентиран към връзката, който улеснява обмена на съобщения между изчислителни устройства в мрежа. Това е най-разпространеният протокол в мрежи, който използва Интернет протокол (IP); заедно те понякога се наричат TCP/IP. TCP взема съобщения от приложение/сървър и ги разделя на пакети, които след това могат да бъдат препратени от устройствата в мрежата – комутатори, рутери, защитни шлюзове – до местоназначението. TCP номерира всеки пакет и го сглобява отново, преди да ги предаде на получателя на приложението/сървъра. Тъй като е ориентиран към връзката, той гарантира, че връзката е установена и поддържана, докато обменът между приложението/сървърите, изпращащи и получаващи съобщението, завърши.

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

■ UDP (User Datagram Protocol):

UDP също е протокол от слой 4, но за разлика от TCP не предоставя потвърждение на изпратените пакети. Следователно, той не е надежден и зависи от протоколите на по-високия слой за същото. Но от друга страна е прост, мащабируем и идва с по-малко разходи в сравнение с TCP. Използва се във видео и гласов стрийминг.

TCP	UDP
Keeps track of lost packets. Makes sure that lost packets are re-sent	Doesn't keep track of lost packets
Adds sequence numbers to packets and reorders any packets that arrive in the wrong order	Doesn't care about packet arrival order
Slower, because of all added additional functionality	Faster, because it lacks any extra features
Requires more computer resources, because the OS needs to keep track of ongoing communication sessions and manage them on a much deeper level	Requires less computer resources
Examples of programs and services that use TCP: - HTTP - HTTPS - FTP - Many computer games	Examples of programs and services that use UDP: - DNS - IP telephony - DHCP - Many computer games

УСУКАНА МРЕЖОВА РАМКА (TWISTED NETWORK FRAMEWORK)

Twisted е framework за писане на асинхронни, управлявани от събития мрежови програми в Python - както клиенти, така и сървъри. В допълнение към абстракциите за системни повиквания от ниско ниво като `select` и `socket`, той също така включва голям брой помощни функции и класове, които правят писането на нови сървъри лесно.

Twisted включва поддръжка за популярни мрежови протоколи като HTTP и SMTP, поддръжка за GUI рамки като GTK+/GNOME и Tk и много други класове, предназначени да направят мрежовите програми лесни. Когато е възможно, Twisted използва средствата за интроспекция на Python, за да спести на клиентския програмист възможно най-много работа. Въпреки че Twisted все още е в процес на работа, той вече може да се използва за производствени системи - може да се използва за извеждане на уеб сървър, пощенски сървър или IRC сървър за броени минути и не изисква почти никаква конфигурация.

Python се поддава на писане на рамки. Python има прост модел на клас, който улеснява наследяването. Той има динамично въвеждане, което означава, че кодът трябва да се приема по-малко. Python също има вградено управление на паметта, което означава, че кодът на приложението не трябва да проследява собствеността. По този начин, когато пише ново приложение, програмист често се оказва, че пише framework, за да улесни писането на този вид приложение.

Twisted еволюира от необходимостта от писане на високопроизводителни оперативно съвместими сървъри в Python и ги прави лесни за използване (и трудни за използване неправилно).

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

Има три начина за писане на мрежови програми:

- 1. Обработване на всяка връзка в отделен процес**
- 2. Обработване на всяка връзка в отделна нишка**
- 3. Използване на неблокиращи системни повиквания, за да се обработят всички връзки в една нишка.**

▪ USENET

Usenet е световна разпространена дискуссионна система, достъпна на компютри. Той е разработен от общата архитектура за комутируема мрежа Unix-to-Unix Copy (UUCP). Потребителите четат и публикуват съобщения (наричани статии или публикации и общо наричани новини) в една или повече категории, известни като дискуссионни групи.

Usenet прилича на система за бюлетини (BBS) в много отношения и е предшественик на интернет форумите, които се използват широко днес. Дискусиите са нишки, както при уеб форумите и BBS, въпреки че публикациите се съхраняват на сървъра последователно. Името идва от термина "потребителска мрежа" ("users network").

- NEWSGROUP – E-MAIL

comp.lang.python

comp.lang.python е отворена (немодерирана) дискуссионна група на Usenet с голям обем за общи дискусии и въпроси относно Python. Можете също да получите достъп до него като пощенски списък чрез **python-list**. Почти всичко, свързано с Python, е дискусия, а групата дори е доста толерантна към отклонения извън темата; имаше забавни дискусии на теми като плаваща запетая, добър софтуерен дизайн и други езици за програмиране като Lisp и Forth.

Повечето дискусии относно **comp.lang.python** са за разработване с Python, а не за разработка на самия интерпретатор на Python. Някои от основните разработчици все още четат списъка. Понякога предложенията на **comp.lang.python** водят до писане на предложение за подобрение, което води до нова функция на Python.

Ако откриете грешка в Python, не я изпращайте на comp.lang.python; подайте доклад за грешка в програмата за проследяване на проблеми.

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

▪ SIMPLE MAIL TRANSFER PROTOCOL (SMTP)

SMTP е протокол, който обработва изпращането на електронна поща и маршрутизирането на електронна поща между пощенските сървъри. Python предоставя модул **smtplib**, който дефинира обект на сесия на SMTP клиент, който може да се използва за изпращане на поща до всяка интернет машина с демон на SMTP или ESMTP слушател.

Ето един прост синтаксис за създаване на един SMTP обект, който по-късно може да се използва за изпращане на електронна поща:

```
import smtplib
smtpObj = smtplib.SMTP( [host [, port [, local_hostname]]] )
```

Подробно описание на параметрите:

host – Това е хостът, който изпълнява вашия SMTP сървър. Можете да посочите IP адрес на хоста или име на домейн като **tutorialspoint.com**. Това е незадължителен аргумент.

port – Ако предоставяте аргумент за хост, тогава трябва да посочите порт, където SMTP сървърът слуша. Обикновено този порт би бил 25.

local_hostname – Ако вашият SMTP сървър работи на вашата локална машина, тогава можете да посочите само localhost от тази опция.

SMTP обектът има метод на екземпляр, наречен **sendmail**, който обикновено се използва за извършване на работата по изпращане на съобщение. Необходими са три параметъра:

Подателят (sender) – низ с адреса на подателя.

Получателите (receivers) – списък с низове, по един за всеки получател.

Съобщението (message) – съобщение като низ, форматиран, както е посочено в различните RFC

Тема 10_1. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул

▪ POP3

За да получите имейл, можете да напишете MUA - Mail User Agent (пощенски потребителски агент) като клиент и да извлечете имейла от MDA - Mail Delivery Agent (агент за доставка на поща) до компютъра или мобилния телефон на потребителя.

Най-често използваният протокол за получаване на поща е POP протокол. Текущият номер на версията е 3, известен като POP3. Python има вграден модул **poplib**, който реализира **POP3** протокол и може да се използва за директно получаване на поща. Протоколът POP3 не получава самото оригинално четливо съобщение, а кодирания текст на съобщението, изпратено от SMTP. За да превърнете текста, получен от POP3, в четим имейл, е необходимо да анализирате оригиналния текст с различни класове, предоставени от имейл модула, и да го превърнете в четим имейл обект. Има две стъпки за получаване на имейл от POP3 сървър в Python. Протоколът POP3 е имейл протокол за изтегляне на съобщения от имейл сървър. Тези съобщения могат да се съхраняват в локалната машина.

▪ Key Points

- POP е интернет стандартен протокол на приложния слой.
- Тъй като POP поддържа офлайн достъп до съобщенията, по този начин изисква по-малко време за използване на интернет.
- POP не позволява търсене.
- За да получите достъп до съобщенията, е необходимо да ги изтеглите.
- Позволява да се създаде само една пощенска кутия на сървър.
- Не е подходящ за достъп до данни извън пощата.

Използвана литература:

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009